

Reporte de desarrollo del proyecto 1: Aplicación chat

Gómez Ferro Ana Luisa

Materia: Modelado y Programación

29 de septiembre de 2026

Resumen

En este proyecto se aborda el diseño e implementación de una aplicación de chat cliente/servidor mediante programación orientada a objetos. Para resolver el problema de la comunicación en tiempo real y la gestión de múltiples usuarios, se desarrolló un programa multihilo en C++ que usa el compilador GCC. Usaremos la librería cJSON para el empaquetado y desempaqueado de la estructura de mensajes, el marco Google Test (GTest) para la automatización de pruebas unitarias para el servidor, el sistema de construcción CMake/Make, y Doxygen para la generación de documentación. Como resultado, se obtuvieron dos ejecutables: un servidor capaz de gestionar múltiples conexiones concurrentes, manejar salas y reenviar mensajes públicos o privados, y un cliente que proporciona una interfaz, mantiene la conexión con el servidor y permite la comunicación en tiempo real.

1. Introducción

El siguiente proyecto tiene como objetivo principal diseñar e implementar una solución integral de chat cliente-servidor utilizando el lenguaje C++ bajo el paradigma orientado a objetos, mediante la metodología del modelo en cascada (que abarca las fases de análisis, diseño, implementación, pruebas y mantenimiento, centrándonos principalmente en las etapas de análisis, diseño e implementación).

El problema central radica en permitir la interacción simultánea entre múltiples clientes independientes a través de un único servidor. La aplicación debe garantizar que los mensajes (públicos y privados) sean enviados correctamente, mantener la existencia de varias salas de conversación y mantener la estabilidad del sistema ante eventos inesperados, como la desconexión repentina de un cliente, el uso de un nombre ya usado, nombres excesivamente grandes, etc. Todo esto, respetando un protocolo de comunicación que debe mantener tanto el cliente como el servidor.

Durante el desarrollo se presentaron varias dificultades, tales como evitar que dos o más hilos de ejecución accedan de forma simultánea a los recursos compartidos (como la lista general de usuarios y las listas de las salas) en el servidor, la creación, uso y correcto cierre de los sockets tanto para el cliente como para el servidor, y el uso de un lenguaje de programación que no conocía, que en este caso es C++, al igual que el uso de otras tecnologías ya mencionadas.

Entre las facilidades del proyecto destaca el uso de la programación orientada a objetos, y la facilidad al estructurar los mensajes del protocolo con la ayuda de cJSON.

2. Análisis y Diseño

El planteamiento del problema a resolver es el siguiente: se requiere una aplicación chat que se componga de un programa servidor y un programa cliente; debe existir una comunicación continua entre varios clientes (o usuarios) a través del servidor, que funcionará de manera concurrente. En esta aplicación los clientes serán capaces de realizar múltiples cosas, las cuales se encuentran descritas en el protocolo del proyecto. Además, para la comunicación entre los clientes y el

servidor se debe usar el formato JSON. Este es un problema ya conocido y que ha tenido varias implementaciones a lo largo del tiempo, e incluso con una complejidad mayor, lo que asegura que el problema se puede resolver mediante la creación de algoritmos, es decir, se sabe que el problema es computable.

Para comprender más a fondo el problema se dividirá en problemas más pequeños. Tenemos que tener dos programas: un servidor que acepte clientes y realice múltiples operaciones, y un cliente que pueda conectarse al servidor y solicitar varias cosas al mismo, además de tener una manera adecuada de mostrar el chat al usuario.

2.1. Diseño del Programa del Servidor

Para el diseño del servidor, siguiendo el principio de responsabilidad única, se dividió el problema en múltiples clases especializadas:

La clase **Servidor** tiene la responsabilidad de administrar la creación del socket pasivo (escucha), vincular la dirección IP y el puerto (**bind**), poner el socket en modo receptivo (**listen**) y aceptar conexiones entrantes (**accept**). La clase Servidor no procesa peticiones de mensajería; al aceptar un cliente mediante **accept()**, instancia un objeto **ManejadorCliente** al que le pasa el socket del cliente y una instancia de la clase **ManejadorEstados** que es compartida por todos, ejecutándola en su propio hilo de ejecución independiente. Esto evita que el bucle principal de red se bloquee con los mensajes de los clientes. Para cerrar el programa, captura la señal **SIGINT** (Ctrl + C) mediante un manejador global en el **main** que invoca el método **detener**, cerrando el socket pasivo y desconectando los sockets de los clientes activos.

La clase **ManejadorCliente** brinda atención exclusiva a un cliente gestionando su flujo completo de vida en su propio hilo. Extrae los mensajes procesando el flujo de bytes entrantes por el socket para fragmentar y retornar las cadenas cJSON recibidas. Utiliza la clase **ConstructorMensajes** para transformar la cadena JSON en una estructura de información (**MensajeProtocolo**). Posteriormente, un despachador de acciones lee la solicitud (**IDENTIFY**, **NEW_ROOM**, **PUBLIC_TEXT**, etc.) y manda a llamar a un método auxiliar especializado (como **mensajePublico** o **crearSala**), los cuales realizan las peticiones o modificaciones necesarias a la instancia de **ManejadorEstados**.

La clase **ManejadorEstados** actúa como el núcleo de datos compartido, almacenando la lista global de clientes activos, sus nombres, sus sockets, sus estados y la estructura de salas con sus respectivos miembros. Al ser un recurso compartido al que múltiples hilos **ManejadorCliente** acceden simultáneamente, todos sus métodos de lectura y escritura están protegidos internamente con exclusión mutua (**std::mutex** / **std::lock_guard**). Además, controla el tiempo de vida de las salas (eliminando una sala automáticamente cuando su número de integrantes llega a cero) y mantiene a los usuarios de la lista global y de las respectivas salas mediante métodos que permiten acceder a ellos.

Antes de llegar a este diseño final evalúe otras alternativas que terminaron siendo inviables por la complejidad que implicaban o por ser muy ineficientes y desordenadas. Al principio se me ocurrió tener una única lista grande guardada en el servidor donde vivieran todos los usuarios con sus sockets, sus datos y las salas a las que pertenecían, y cada vez que un cliente quería hacer algo (por ejemplo, mandar un mensaje a una sala) tenía que recorrer a todos los clientes registrados, checar uno por uno quiénes estaban metidos en esa sala específica y enviarles el mensaje. Aunque parecía fácil de pensar al inicio, me di cuenta de que era mala idea por varias razones: hacía búsquedas innecesarias (si había 100 usuarios conectados y solo 3 pertenecían a la sala, tenía que revisar a los 100 de todos modos) y, debido a que todos los hilos accederían todo el tiempo a ella, seguramente se volvería un proceso muy lento; además, mezclaba responsabilidades porque un solo cliente no debería tener la carga de andar buscando y organizando a los demás. Por eso descarté esa idea antes de ponerme a programar y decidí crear la clase **ManejadorEstados**.

Por otro lado, intenté usar en **ManejadorEstados** el patrón Singleton, ya que solo existe una instancia de él en todo el servidor. Pero me di cuenta de que no era buena idea porque seguirlo

implicaba esconder el constructor y obligar a todo el código a buscar una instancia global estática. Por eso preferí que el servidor creara la instancia al arrancar y se la pasara como referencia a cada cliente cuando se conecta, lo que era mucho más fácil y limpio.

2.2. Diseño del Programa del Cliente

Al igual que con el servidor, el diseño del cliente se planificó siguiendo el principio de responsabilidad única, pero además, dadas las características del programa, se decidió usar el patrón Modelo-Vista-Controlador (MVC) para la organización de las clases:

El Modelo está representado por la clase **AlcanceCliente**, componente de la arquitectura encargado de representar y salvaguardar el estado global del cliente en memoria. Mantiene la información del usuario (nombre y estado), la lista de usuarios conectados al servidor y la estructura de las salas de chat a las que pertenece el cliente (con sus respectivos miembros). Como lo especifica el patrón de diseño, el modelo no conoce detalles sobre la interfaz gráfica ni sobre el socket de red. Su única labor es mantener en sincronía con el servidor la información local y ofrecer métodos para consultar o actualizar dichos datos.

La Vista se implementó en la clase **VistaCliente**, encargada de la presentación e interfaz de usuario en la consola. Únicamente tiene métodos de salida que imprimen las acciones que pasan en el chat. Tiene métodos dedicados para cada respuesta o notificación que el servidor pueda generar (mensajes recibidos, notificaciones de unirse/abandonar sala, alertas de cambio de estado), además de algunos métodos para imprimir cosas extra que no vienen especificadas en el protocolo con el fin de hacer la experiencia del cliente más óptima. La vista jamás lee del socket, ni analiza JSON, ni toma decisiones; simplemente recibe datos procesados y los muestra de forma estética en la pantalla del usuario.

El Controlador se encuentra estructurado en la clase **ControladorCliente**, la cual orquesta la interacción entre el Modelo, la Vista y la clase encargada de recibir las respuestas del Servidor. Se encarga de la captura y análisis manteniendo el bucle principal que lee las entradas del teclado del usuario y analiza la sintaxis de los comandos en la consola (por ejemplo: `/msg`, `/unirse`, `/invitar`, `/status`). También actúa como intermediario: si el usuario solicita ver una información local (como consultar sus salas actuales), el Controlador consulta directamente al Modelo (**AlcanceCliente**) y le pasa los datos a la Vista (**VistaCliente**) para que los imprima. Si la acción requiere comunicarse con el servidor, invoca a la clase auxiliar **PedirServ** para que empaque y envíe la petición en formato cJSON.

Para llevar un control adecuado de las solicitudes y respuestas con el servidor, se crearon dos clases especializadas que interactúan con la arquitectura MVC. Por un lado, **PedirServ** es utilizada por varios métodos del Controlador para la salida de datos, traduciendo cada petición del usuario a un paquete cJSON enviado al socket. Por otro lado, **RecibirServ** ejecuta su propio hilo de lectura, escuchando continuamente la llegada de paquetes JSON desde el servidor, descomponiendo la respuesta, actualizando directamente el Modelo (**AlcanceCliente**) y notificando al Controlador (que posteriormente notificará a la Vista) para proyectar la actualización en pantalla en tiempo real.

Antes de llegar a este diseño evalúe otras alternativas que terminaron siendo inviables por la complejidad que implican o por ser muy desordenadas. Al inicio quería una sola clase que se encargara de leer de la terminal, procesar los sockets y mostrar los mensajes. Pero lo descarté cuando me percaté del problema que había por el bloqueo entre la recepción de mensajes del servidor y la recepción de mensajes del cliente. Si una sola clase atiende al usuario y al servidor al mismo tiempo, el programa se quedaba congelado esperando a que la persona presione Enter en la terminal, y los mensajes enviados por otros usuarios no se mostraban en pantalla hasta que el usuario escribiera algo, arruinando la experiencia en tiempo real del chat. Además, decidí usar el patrón MVC porque me permitía abordar el problema de una forma mucho más organizada, lo que me ayuda a hacer el programa mucho más rápido y a incluir cosas extra. Como la Vista (**VistaCliente**) no usa sockets ni lógica de comandos, me permitía incluir detalles extra que

aportaran estilo al programa sin preocuparme por la lógica detrás del cliente; incluso pensé en tener una vista en terminal de forma temporal y luego incluir una interfaz gráfica (aunque no me dio tiempo).

3. Implementación

Para el desarrollo del sistema se utilizó C++ como lenguaje de programación principal, aprovechando el paradigma orientado a objetos para el modelado de clases y estructuras. Para el formateo e intercambio de mensajes cliente-servidor se integró la librería `cJSON`, la cual permitió serializar y deserializar los paquetes de información de manera estructurada. En la infraestructura de construcción se utilizó `CMake` (versión 3.10 o superior) junto con `Make` y el compilador `g++` (con soporte para el estándar C++17), complementado con el framework `Google Test (GTest)` para la automatización de pruebas unitarias durante cada proceso de compilación para el programa del Servidor, y por último, se usó la herramienta `Doxygen` para la generación formal de la documentación del código.

Para ejecutar la aplicación en cualquier equipo con entorno Linux, es necesario contar con los paquetes de desarrollo básicos (`gcc-c++`, `cmake`, `make` y `gtest-devel`). El proceso de compilación se realiza desde la terminal ubicándose en la raíz del proyecto y ejecutando los comandos `cmake -B build` seguido de `cmake --build build`. Durante este proceso, `CMake` compila y ejecuta automáticamente la suite de pruebas unitarias del servidor (`test_servidor`); si todas las pruebas pasan de manera exitosa, se generan los dos binarios ejecutables finales en la carpeta `build`: el servidor (`./build/servidor`) y el cliente (`./build/cliente`).

4. Conclusiones

El desarrollo de este proyecto representó un aprendizaje en torno a cómo construir aplicaciones que se comunican en tiempo real. Aprendí a manejar la programación en red a través de sockets y a manejar la programación concurrente mediante el uso de hilos de ejecución. Uno de los aspectos más difíciles fue el proteger correctamente el acceso a las estructuras de datos compartidas utilizando mecanismos de exclusión mutua, ya que un mal manejo hacía fallar a todo el servidor. Asimismo, visualizar el funcionamiento global del sistema desde las primeras etapas y planificar correctamente la interacción entre las clases fue un desafío que requirió replantear el diseño varias veces antes de siquiera empezar a programar.

Durante el proceso identifiqué decisiones de diseño que pude haber mejorado. Por ejemplo, en el lado del servidor cometí el error de almacenar una copia completa de cada cliente por cada lista en la que participaba (una en la lista general de usuarios y otra independiente dentro de cada sala a la que se unía). Esto complicó considerablemente la sincronización de estados, pues actualizar un dato implicaba modificar múltiples duplicados. Sin embargo, una vez identifiqué ese error, para el lado del cliente lo corregí en la clase `AlcanceCliente`, haciendo que las salas almacenaran únicamente referencias directas a una sola instancia del usuario, lo que simplificó notablemente la gestión de la memoria y la actualización de los datos. De igual forma, pude haber optimizado mejor mis tiempos para desarrollar pruebas unitarias más extensas, e incluir también para las clases del cliente y agregar algunas funcionalidades extra que tuve que omitir.

Por último, comprendí la importancia de los patrones de diseño como el Modelo-Vista-Controlador (MVC), el cual demostró ser muy útil al momento de planificar y escribir todo el programa. Si decidiera expandir el sistema a futuro, propondría implementar una interfaz gráfica de usuario en el cliente aprovechando que la separación de clases actual permite sustituir la vista sin alterar la lógica del protocolo, además de otras funciones como el poder guardar los mensajes y las salas para que el cliente no pierda la información al desconectarse.

Referencias

- [1] F. Charte, *Programación con sockets (C++) - Sockets IP (raw sockets)*, 2023. [En línea]. Disponible en: https://www.youtube.com/watch?v=Qfue_nWf0-c
- [2] *Programación de Sockets TCP en C++*, Vídeo de YouTube. [En línea]. Disponible en: <https://www.youtube.com/watch?v=yRzsLVMcR7c>
- [3] *Tutorial de Sockets y Concurrencia en C++*, Vídeo de YouTube. [En línea]. Disponible en: <https://www.youtube.com/watch?v=K8ATqLVFfpo>
- [4] Tutorialspoint, *C++ Socket Programming*. [En línea]. Disponible en: https://www.tutorialspoint.com/cplusplus/cpp_socket_programming.htm
- [5] H. Lu, *tcp-server-*, Repositorio de GitHub. [En línea]. Disponible en: <https://github.com/HolyLu/tcp-server->
- [6] Wikibooks, *Programación en C/Sockets - Cliente*, 2023. [En línea]. Disponible en: https://es.wikibooks.org/wiki/Programaci%C3%B3n_en_C/Sockets#Cliente_2
- [7] Documento compartido de apoyo. [En línea]. Disponible en: <https://share.google/xJ4dvtFrhjUvvU0b0>